# Machine Learning Fundamentals in Python
# -
# Supervised Learning

Martin Řeřábek

ZHAW

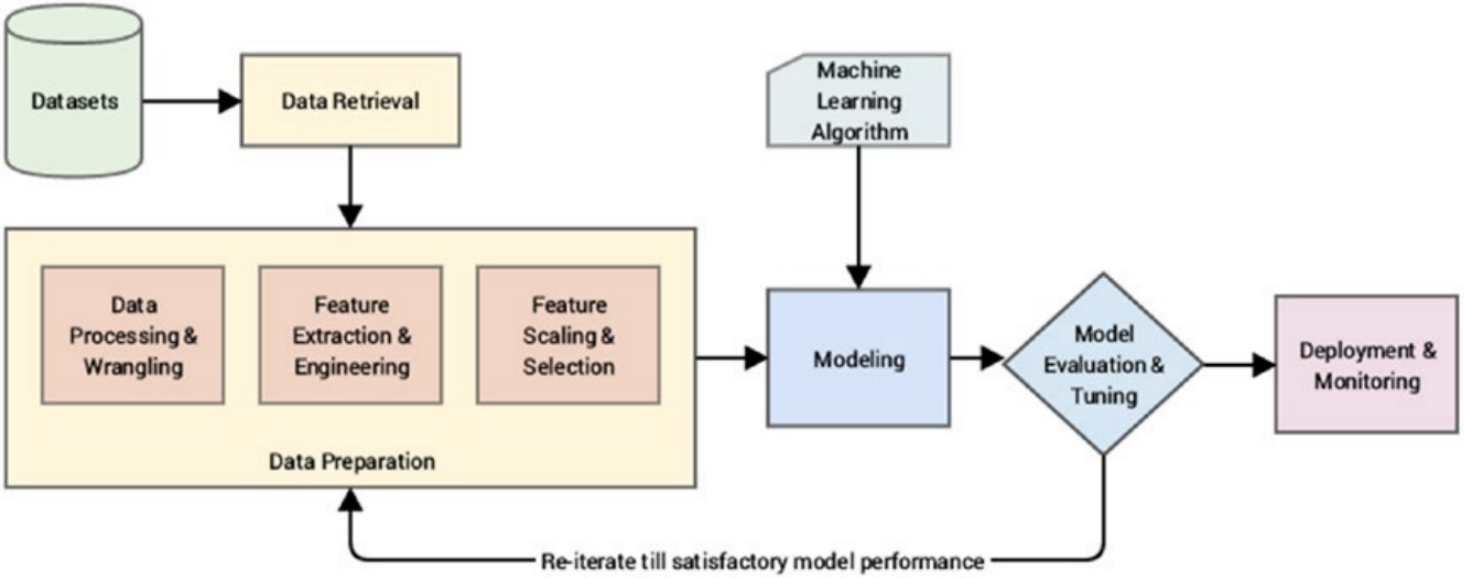
*rera@zhaw.ch*

week 2

# Overview


1. ML General Pipeline
2. Basic Stages of ML
3. Problem Statement Fomulation
4. Basic ML Algorithms
5. Project and Group Work

# ML General Pipeline



# Goal of Supervised Learning

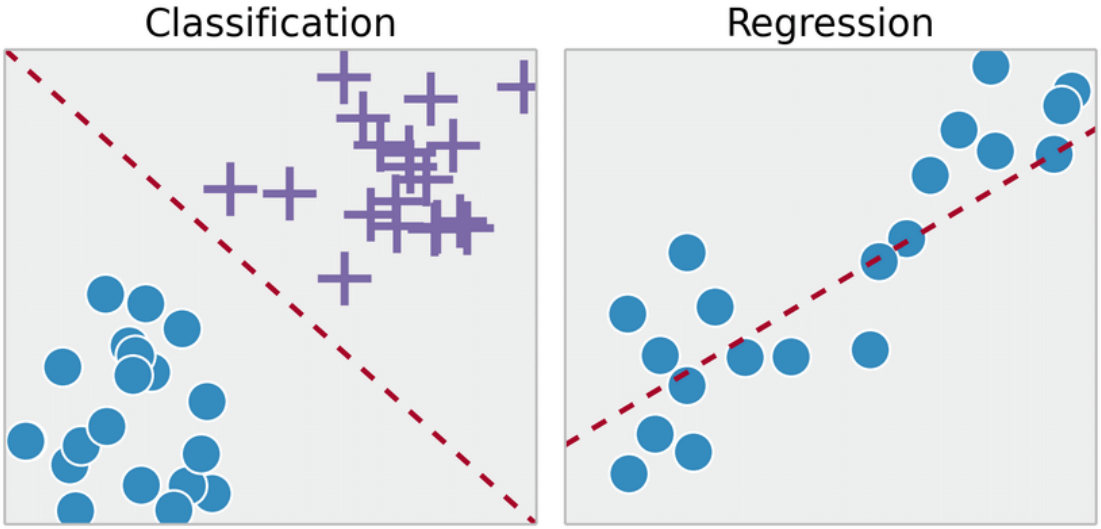
The goal in supervised learning is to make predictions from data

# Three Stages of Machine Learning

- Machine learning field
  - Statistics
  - Mathematics
  - Optimization
  - Linear algebra
  - ...and more
- Simple three-stage paradigm
  - Representation
    - Problem definition
    - What ML algorithm to use to solve our problem?
  - Evaluation
    - Judging criteria to measure the performance of predictive models
    - Define loss/cost function
  - Optimization
    - Search through model parameters to find those giving optimal evaluation criteria value

# Supervised Learning - Problem Formulation I

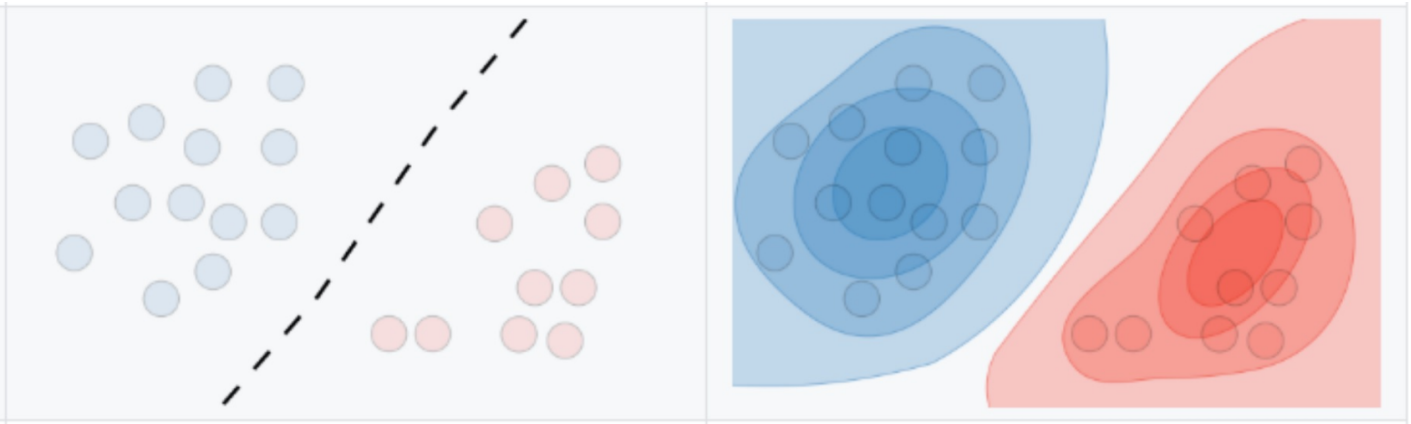
- Set of data points $x_1, ..., x_m$, independent/explanatory, variables, features
- Set of outcomes $y_1, ..., y_m$, dependent/response variable, target
- Given a set of data points associated to a set of outcomes, we want to build a model that learns how to predict y from x.
- Type of prediction/predictive models
  - Regression - continuous outcome, what temperature will be tomorrow
  - Classification - class - will be hot or cold tomorrow



# Supervised Learning - Problem Formulation II

Type of model

- Discriminative - learning decision boundary
- Generative - learning probability distribution of the data



# Supervised Learning - Problem Formulation III

- Training data in pairs of inputs $(\boldsymbol{x}, y), \boldsymbol{x} \in \mathcal{R}^d$
- Training data - $D = \{(\boldsymbol{x}_1, y_1), \ldots, (\boldsymbol{x}_m, y_m)\} \subseteq \mathcal{R}^d \times \mathcal{C}$
  - $\mathcal{R}^d$ is the d-dimensional feature space
  - $\boldsymbol{x}_i$ is the input vector of the $i$-th sample
  - $y_i$ is the label of the ith sample
  - $\mathcal{C}$ is the label space
- The data points drawn from some distribution $P(X, Y)$

# Supervised Learning - Problem Formulation IV

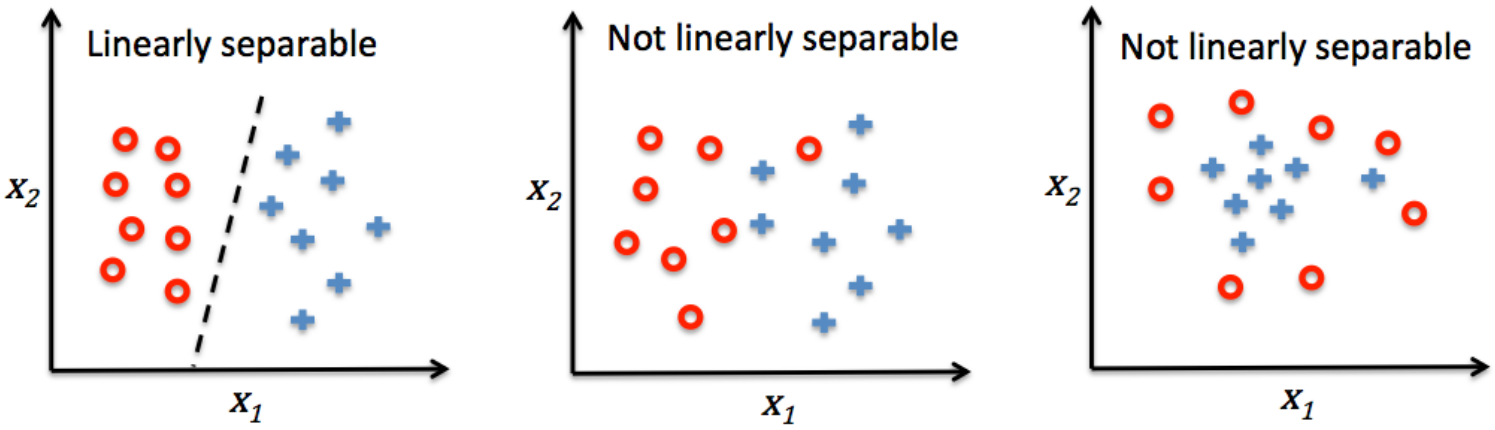
- Goal of Supervised Machine Learning
  - To learn a function $h$ such that for a new pair of input data, we get $h(\boldsymbol{x}) = y$ with high probability, i.e. $h(\boldsymbol{x}) \approx y$
  - We approximate an unknown target function $h$ that can best map inputs $(\boldsymbol{x})$ to outputs $y$ on all possible observations from the dataset
  - Also called function approximation
  - The approximated function is then tested on unseen data. i.e. it should be able to make predictions on the new data.

# Representation - Hypothesis Selection I

- Representation - basic definition
  - Hypothesis $h$ is a single candidate model that maps features to the target
  - Hypothesis space $H$ is a complete range of models and their possible parameters that can be used to model the data
- Specifying $H$
  - Set of function hypotheses $H = \{h | h : \boldsymbol{x} \rightarrow y\}$
  - Make assumptions about the type of problem we are trying to learn
    - Regression, classification, decision tree, NN
    - What type of function we look for?
  - The choice of algorithm and its configuration involves choosing a hypothesis space that is believed to contain a hypothesis that is a good or best approximation for the target function
  - Very challenging, thus more efficient and common is to spot-check a range of different hypothesis spaces

# Representation - Hypothesis Selection II

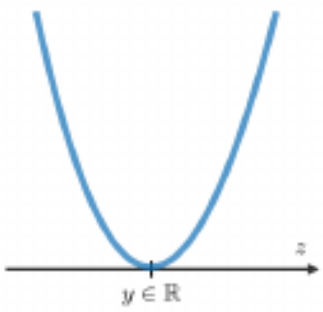
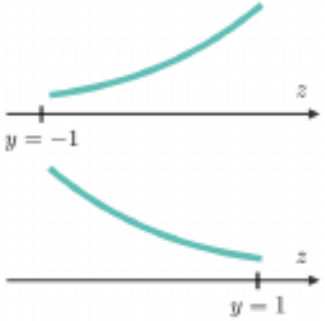
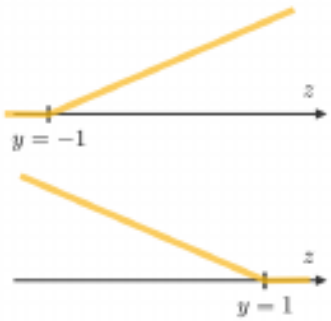
- No Free Lunch Theorem
  - One single ML algorithm does not solve everything
  - A successful ML algorithm must make assumptions



# Evaluation - Loss and Cost Function

- Learning $h(\boldsymbol{x})$ has two steps
  - Select $H$, i.e. the set of functions we can use to learn
  - Find the best function $h(\boldsymbol{x})$ within this class $H$
- Learning process - Optimization problem
  - Minimize the error between predicted data and true data
  - Find $h(\boldsymbol{x})$ which makes smallest number of errors within our training data
- How?
  - What means that one function is better than another?
  - Define loss error and cost function
  - Squared loss, absolute loss, ...
- Loss and cost function
  - The cost function $J$ is used to assess the performance of a model as sum of losses/errors for all samples
  - The loss function $L$ measures errors between predicted and target value per sample
  - $J(w) = \sum_{i=1}^{m} L(h_w(\boldsymbol{x}_i), y_i)$

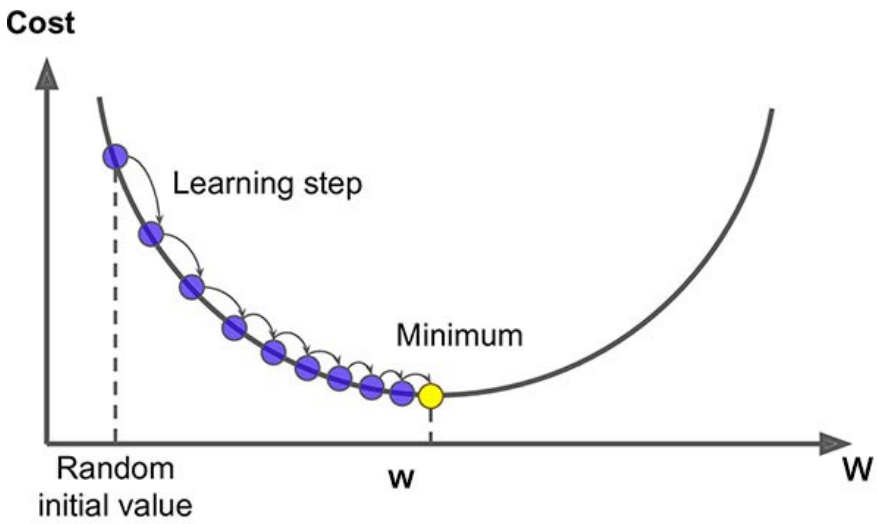
# Evaluation - Loss Function Examples

| **Least squared** | **Logistic** | **Hinge** |
|---|---|---|
| $\frac{1}{2}(y-z)^2$ | $\log(1+\exp(-yz))$ | $\max(0,1-yz)$ |
|   |   |   |
| Linear regression | Logistic regression | SVM |

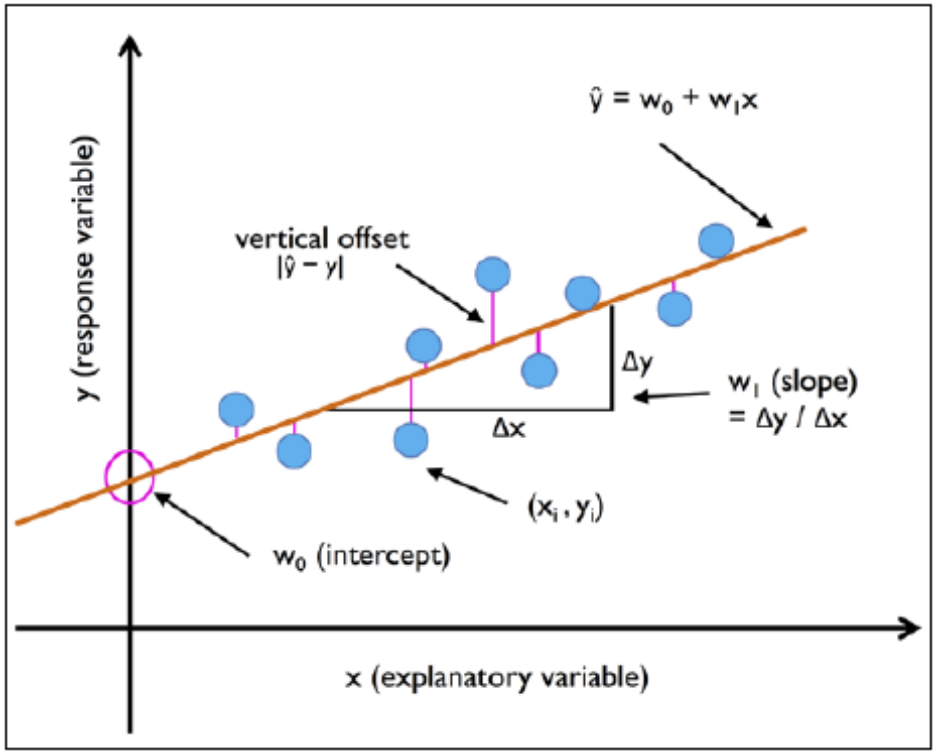
* for a simplicity here $z = \boldsymbol{w}^T\boldsymbol{x}$

# Optimization - Gradient Descent

- Gradient Descent
  - Basic idea of minimizing/maximizing a function is that you differentiate the function and find the point where the gradient is zero
  - Gradient descent = process in which we calculate the gradient of the function we want to optimize at each point and then keep moving in the direction of negative gradient values.
  - Moving = update the values of $w$ according to the calculated gradient
  - Importance of $\alpha$
  - Calculate the gradient of the cost function with respect to each component of the parameter
  - $\boldsymbol{w} \leftarrow \boldsymbol{w} - \alpha \nabla J(\boldsymbol{w})$
  - $w_1 \leftarrow w_1 - \alpha \frac{\partial}{\partial w_1} J(w_1))$

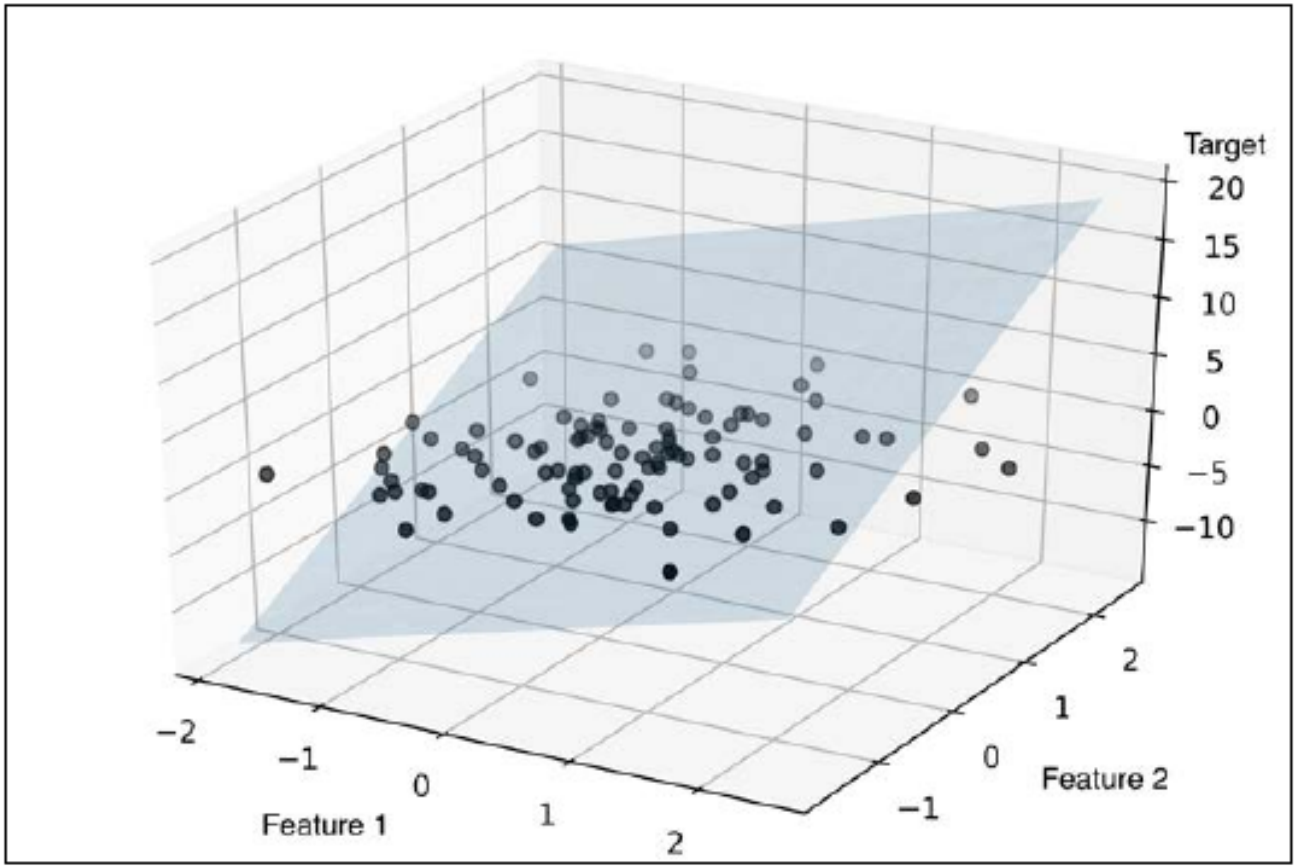


# Linear Regression




- $h_w(x) = \hat{y} = w_0 + w_1 x$
- Best-fitting / regression line
- Vertical lines / offsets / residuals—the errors of our prediction
- $J(w) = \frac{1}{2m} \sum_{i=1}^{m} (y_i - \hat{y}_i)^2$

# Multiple Linear Regression




- $h_w(x) = w_0 x_0 + w_1 x + \cdots + w_m x_m = \boldsymbol{w}^T \boldsymbol{x}$

# Logistic Regression I

- Logistic regression is an algorithm that is used in solving classification problems
- Special case of linear regression where the target variable is categorical in nature, we predict the probability that $y = 1$ on input $\boldsymbol{x}$
- Represented by sigmoid function
- Sigmoid/logistic function

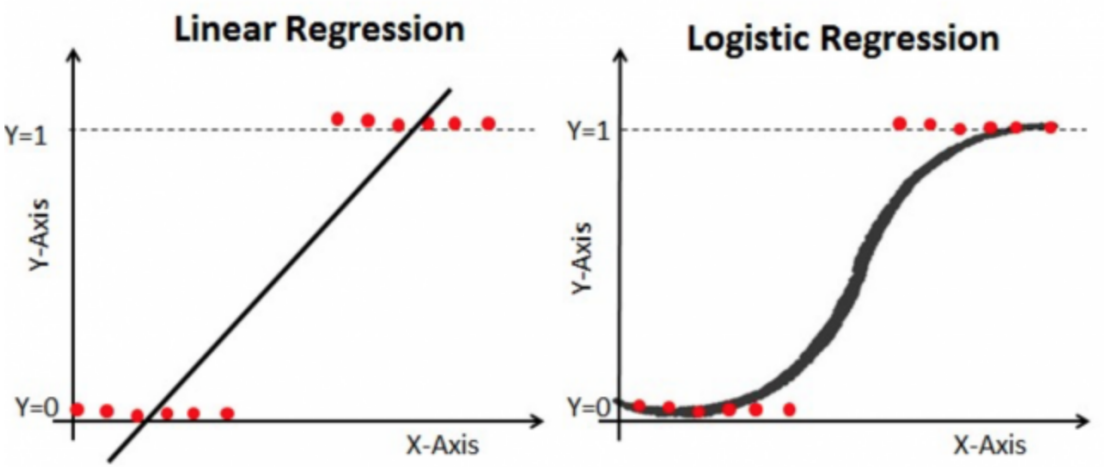
$$\sigma(t) = \frac{1}{1-\exp^{-t}}$$

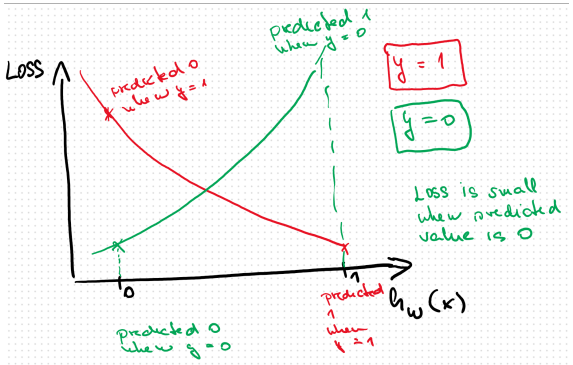
- Hypothesis

$$h_w(x) = \boldsymbol{w}^T \boldsymbol{x} = \frac{1}{1-\exp^{-w^T x}},\ 0 \leq h_w(x) \leq 1$$

- $L(h_w(x_i), y_i) = \begin{cases} -\log(h_w(x_i)) & \text{if } y_i = 1 \\ -\log(1 - h_w(x_i)) & \text{if } y_i = 0 \end{cases}$

# Logistic Regression II



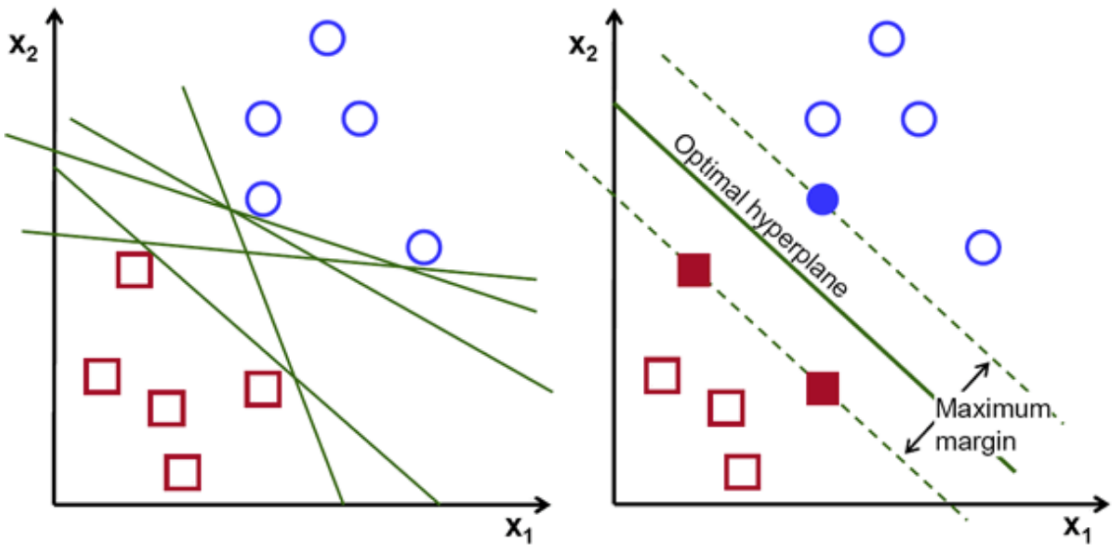



# SVM I

- The support vector machine is a model used for both classification and regression problems though it is mostly used to solve classification problems
- Based on geometrical properties of the data while logistic regression is based on statistical approaches
- Works well with unstructured and semi-structured data - text/images while logistic regression works with already identified independent variables
- Risk of overfitting is less in SVM compared to logistic regression

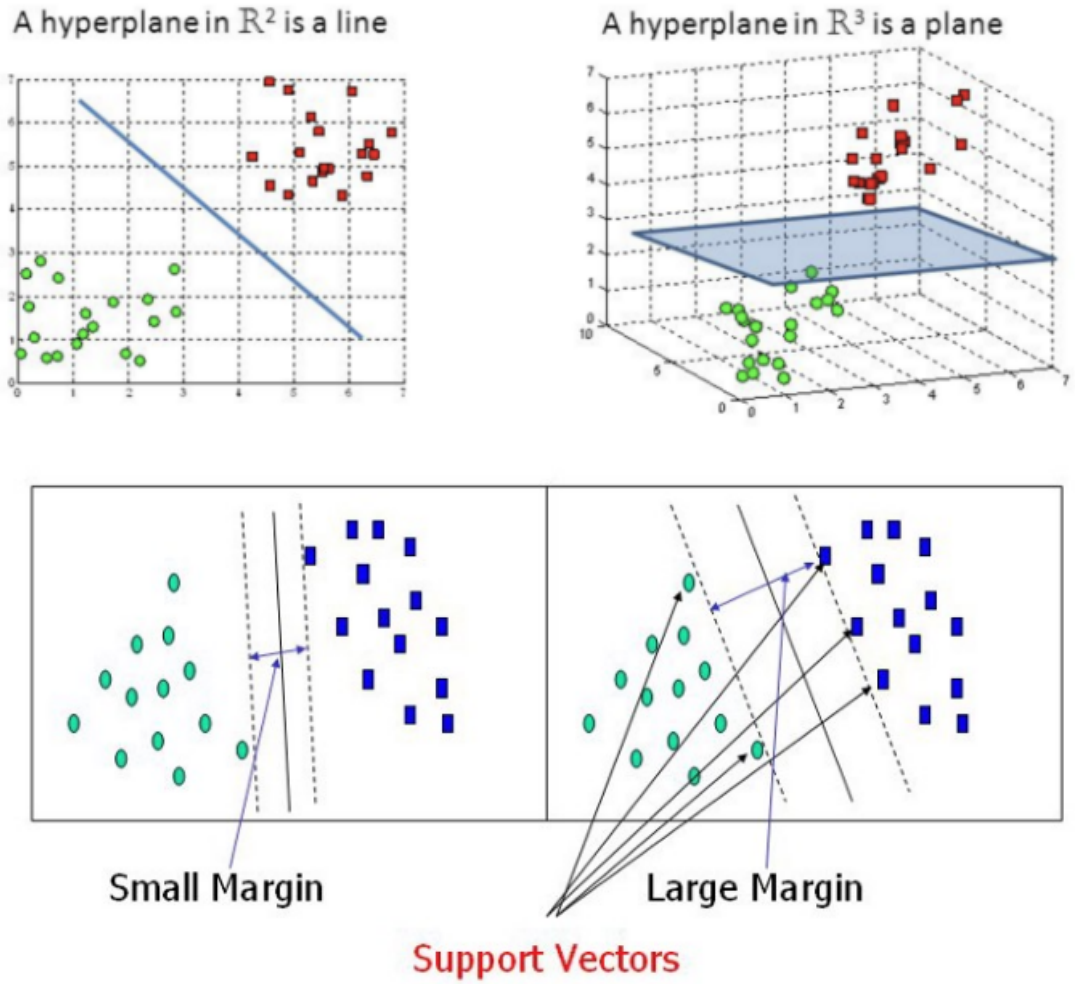
# SVM II

- $h_w(x) = \begin{cases} 1 & \text{if } \boldsymbol{w}^T \boldsymbol{x} >= 0 \\ 0 & \text{otherwise} \end{cases}$
- $L(h_w(x_i), y_i) = \begin{cases} \max(0, 1 - h_w(x_i)) & \text{if } y_i = 1 \\ \max(0, 1 + h_w(x_i)) & \text{if } y_i = 0 \end{cases}$
- Goal to find optimal hyperplane to separate data



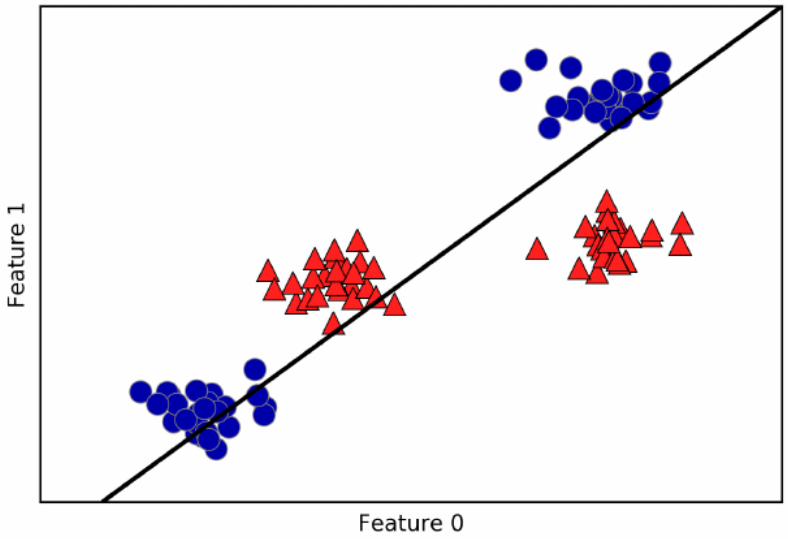
# SVM - hyperplane

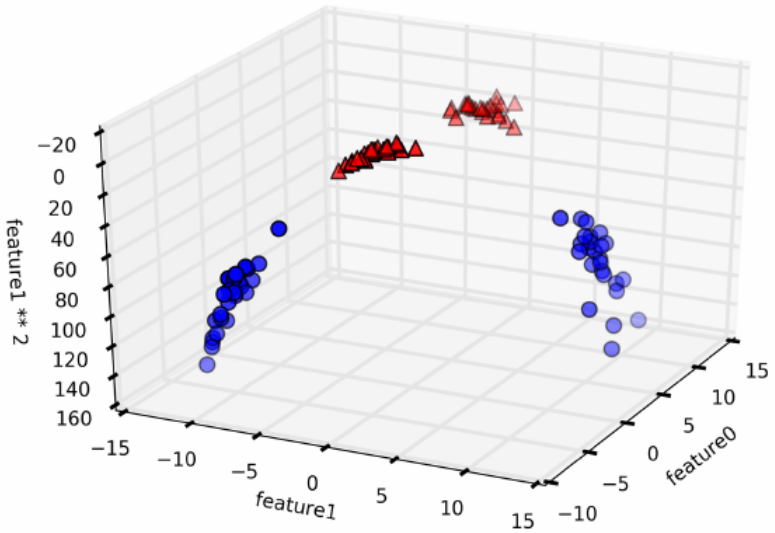
- A hyperplane in an n-dimensional Euclidean space is a flat, n-1 dimensional subset of that space dividing the space into two disconnected parts



# SVM - kernel trick

- In case of no linear separation of data
- We can classify data by adding an extra dimension to it so that it becomes linearly separable and then projecting the decision boundary back to original dimensions using mathematical transformation
- Finding the correct transformation for any given dataset = NOT easy
- Kernel = a function used to map a lower dimensional data into a higher dimensional data



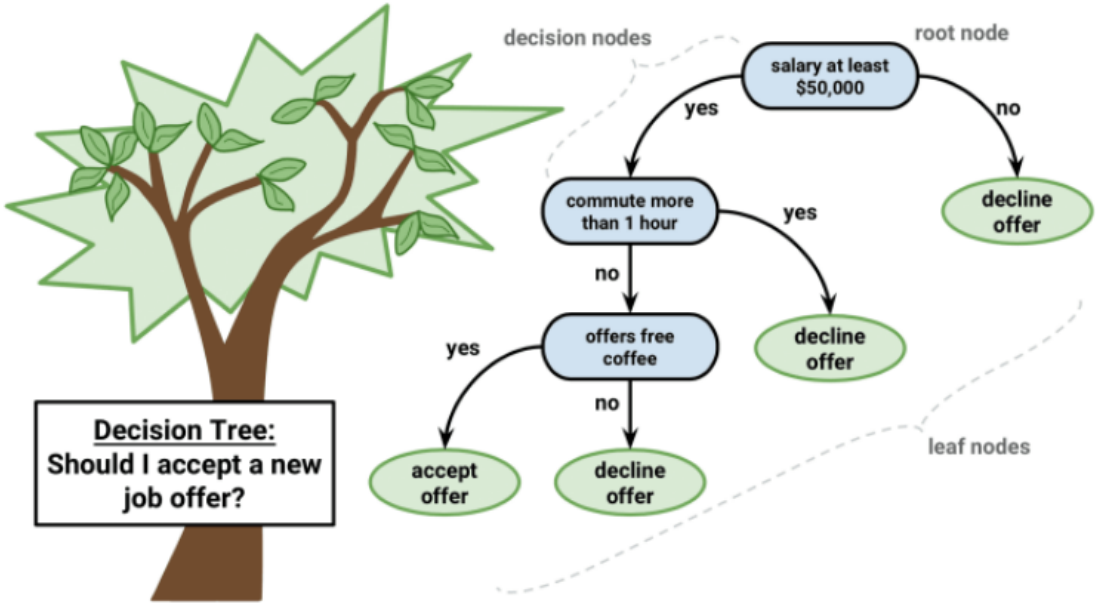



# When to use SVM and Logistic Regression

- Depends on the number of training data/features
- n = number of features, m = number of training examples
  - n is large (1–10,000) and m is small (10–1000)
    - use logistic regression or SVM with a linear kernel
  - n is small (1–10 00) and m is intermediate (10–10,000)
    - use SVM with (Gaussian, polynomial etc.) kernel
  - n is small (1–10 00), m is large (50,000–1,000,000+)
    - first, manually add more features and then use logistic regression or SVM with a linear kernel
- General recommendation = use logistic regression to get base model, in case of failing use SVM without a kernel. They have similar performance but depending on your features, one may be more efficient than the other.

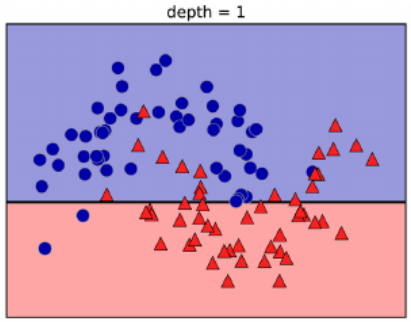
# Decision Trees and Ensemble Methods - Random Forest

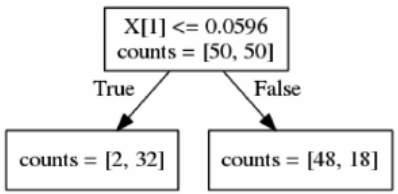
- Widely used models for classification and regression tasks
- They learn a hierarchy of if/else questions, leading to a decision
- Learning a decision
  - learning the sequence of if/else questions that gets us to the true answer most quickly
  - in ML these questions are called tests
  - data not in form of binary features
  - tests on continuous data consists of "Is feature i larger than a value?"

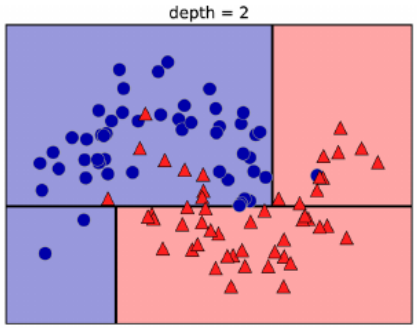


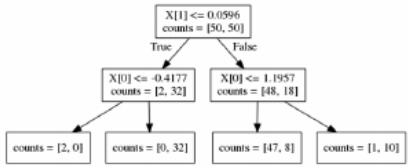
# Decision Trees

- Define loss by measuring purity of leaf nodes
- A leaf of the tree that contains data points that all share the same target value is called pure
- Control depth - full tree very complex and highly overfit to the training data

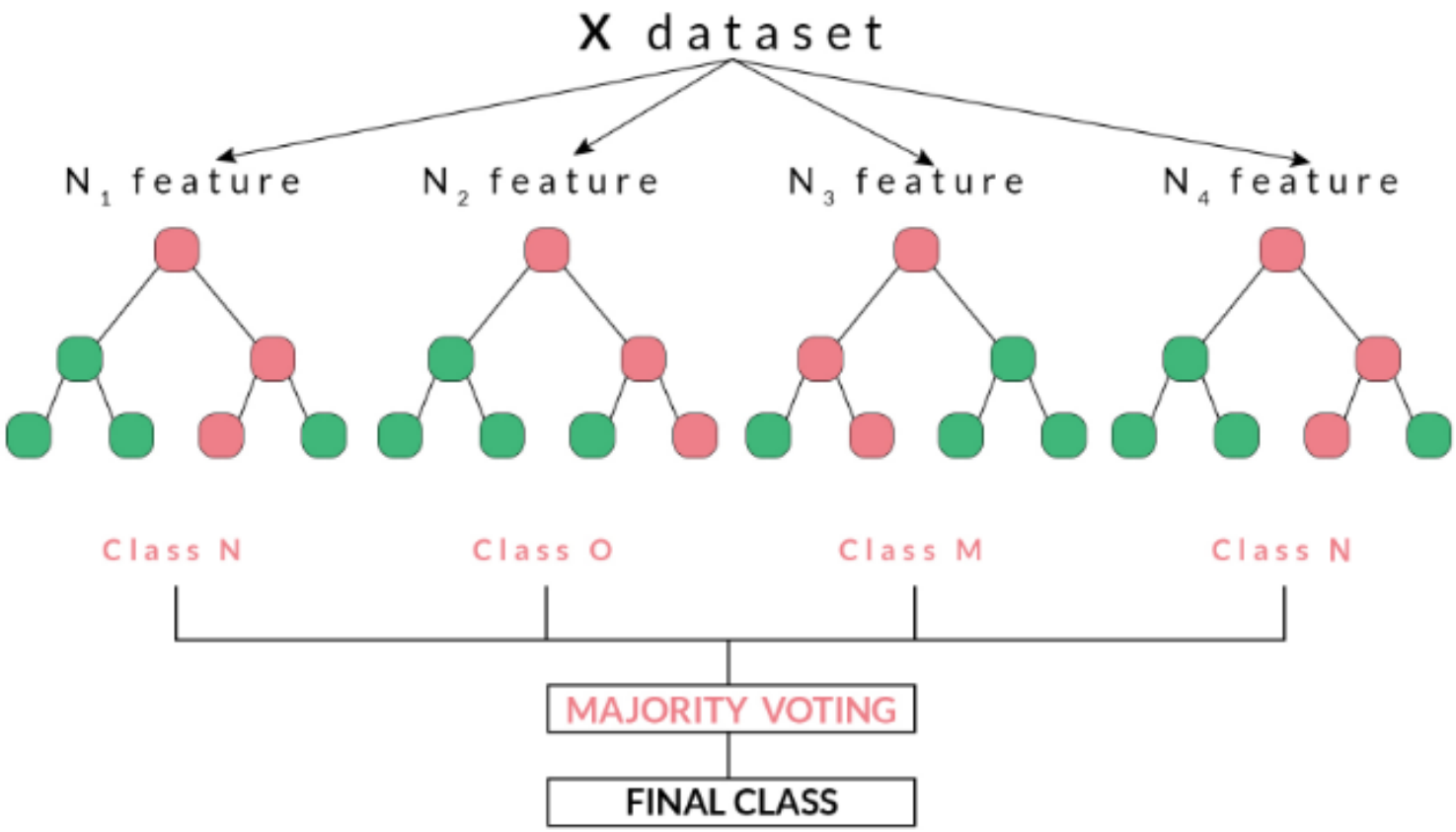


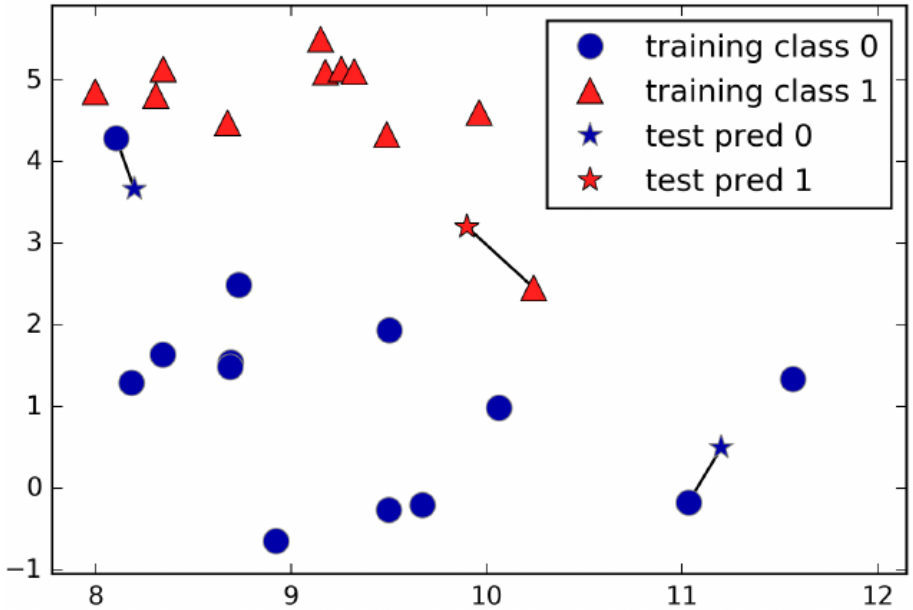



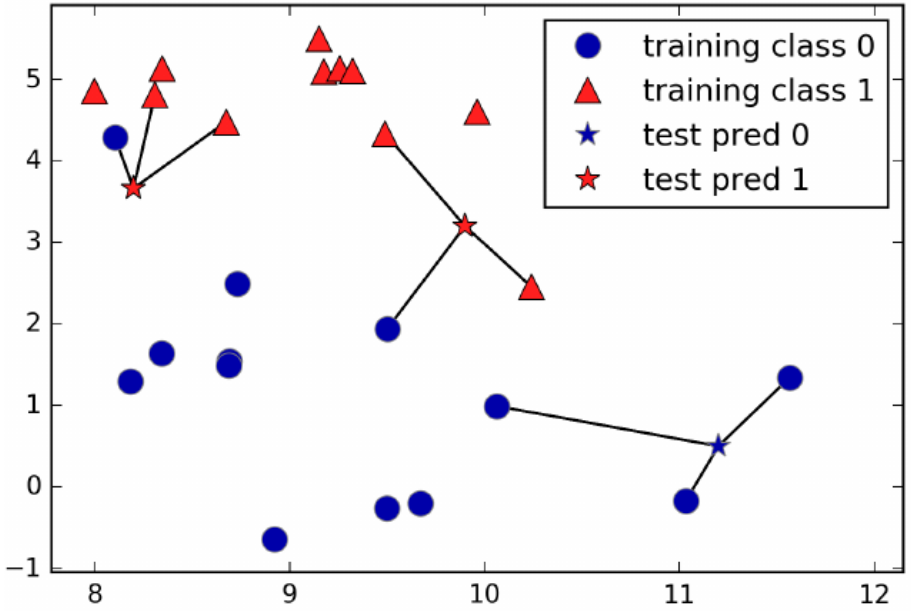
# Ensemble Methods - Random Forest



# KNN

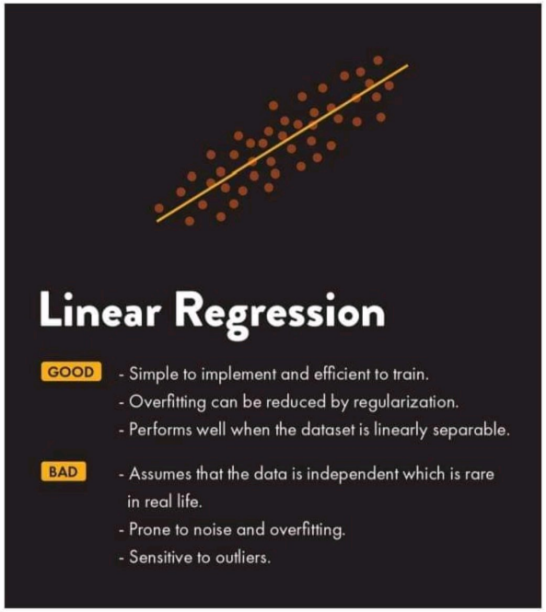
- Building the model consists only of storing the training dataset
- Making a prediction for a new data point, the algorithm finds the closest data points in the training dataset—its "nearest neighbors."
- Parameters - distance measure, number of points

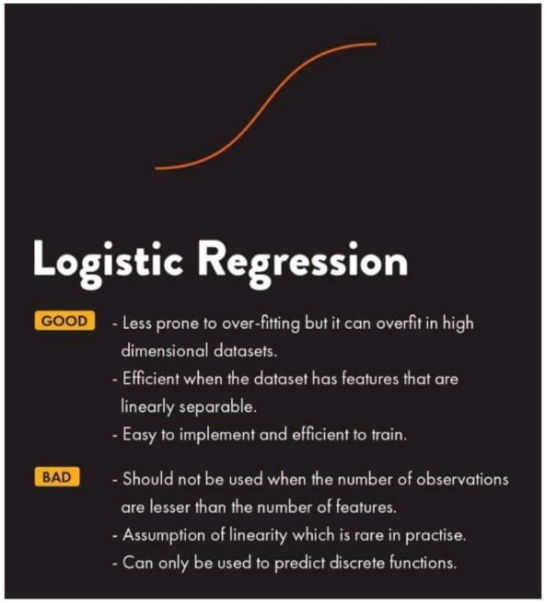


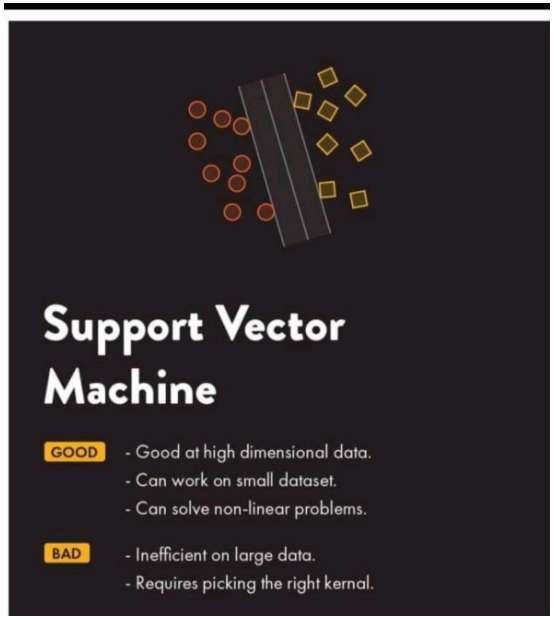



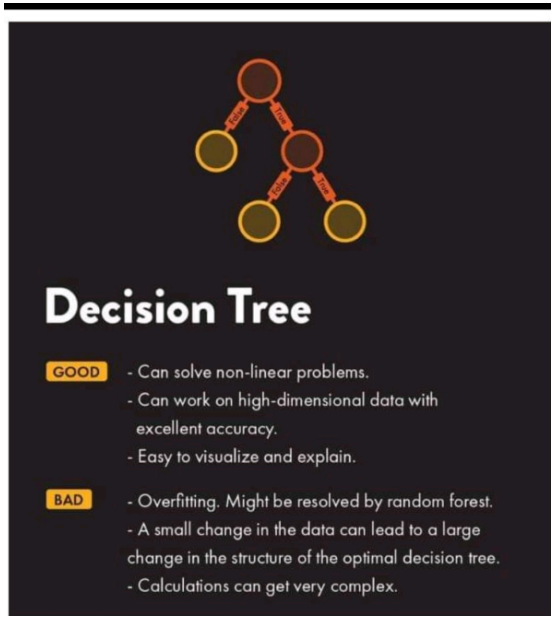
# Other supervised models

- Naive Bayes Classifiers
- Neural Networks


Linear Regression
GOOD
- Simple to implement and efficient to train.
- Overfitting can be reduced by regularization.
- Performs well when the dataset is linearly separable.
BAD
- Assumes that the data is independent which is rare in real life.
- Prone to noise and overfitting.
- Sensitive to outliers.


Logistic Regression
GOOD
- Less prone to over-fitting but it can overfit in high dimensional datasets.
- Efficient when the dataset has features that are linearly separable.
- Easy to implement and efficient to train.
BAD
- Should not be used when the number of observations are lesser than the number of features.
- Assumption of linearity which is rare in practise.
- Can only be used to predict discrete functions.


Support Vector Machine
GOOD
- Good at high dimensional data.
- Can work on small dataset.
- Can solve non-linear problems.
BAD
- Inefficient on large data.
- Requires picking the right kernal.


Decision Tree
GOOD
- Can solve non-linear problems.
- Can work on high-dimensional data with excellent accuracy.
- Easy to visualize and explain.
BAD
- Overfitting. Might be resolved by random forest.
- A small change in the data can lead to a large change in the structure of the optimal decision tree.
- Calculations can get very complex.

K Nearest Neighbour
GOOD
- Can make predictions without training.
- Time complexity is O(n).
- Can be used for both classification and regression.
BAD
- Does not work well with large dataset.
- Sensitive to noisy data, missing values and outliers.
- Need feature scaling.
- Choose the correct K value.

# Projects and Group Work

- Group projects vs individual projects - decide by next lecture
- Data - use toy datasets or your own
- Content
  - Goal definition - goes together with data selection
  - Data description, basic data exploration (what features), visualization = exploratory analysis
  - Data pre-processing - scaling, feature selection, feature engineering
  - Model selection - choose simple method as base model and try to improve its results by selecting another model or by tuning its parameters
  - Model training
  - Model evaluation - select appropriate method to evaluate your model
  - Prepare colab notebook and present it - length of the presentation will depend on number of projects - approximately 15 minutes